OpenZeppelin | security

Uniswap v4 Periphery Permissioned Pools Force-Exit Fix Review



September 18, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Security Model and Trust Assumptions 6

Privileged Roles 6

Medium Severity 7

M-01 Recipients Can Block Force-Exit By Leaving an Unsettled Delta During Delivery 7

Notes & Additional Information 8

N-01 Inconsistent Order of Operations When Clearing a Subscriber 8

Client Reported 8

CR-01 Liquidity Can Be Added Against an Unverified or Undeployed Permissions Adapter 8

Conclusion 10

Appendix 11

Issue Classification 11

Summary

Type	DeFi
Timeline	From 2026-09-04 To 2026-09-04
Languages	Solidity

Total Issues	3 (2 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	1 (1 resolved)
Low Severity Issues	0 (0 resolved)
Notes & Additional Information	1 (0 resolved)
Client Reported Issues	1 (1 resolved)

Scope

OpenZeppelin performed a review of the `Uniswap/v4-periphery` repository at the `c9eaa82e` commit, which remediates a previously reported issue that allowed a position's subscriber to block the force-exit of that position. The review covered the changes relative to the `dce236d4` commit.

The reviewed changes were later published in pull request #602, together with the fix for the medium severity issue identified in this report. The fix for a separate issue reported by the Uniswap team during the engagement was published in pull request #603, and is included in this report as a client-reported issue.

In scope was the following file:

```
v4-periphery/src
└── hooks
    └── permissionedPools
        └── PermissionedPositionManager.sol
```

The inherited `PositionManager` and `Notifier` contracts were reviewed only to the extent that they interact with the changes.

System Overview

A permissioned token is exposed to Uniswap v4 through a `PermissionsAdapter`, an ERC-20 wrapper whose owner controls who may trade and provide liquidity. Liquidity positions in pools containing a verified adapter are managed by `PermissionedPositionManager`, which disables position transfers and adds the `unwindPosition` function. This function is a compliance force-exit: the owner of either adapter in the pool can burn a liquidity provider's position and route each currency to the liquidity provider, falling back to the adapter owner and finally to an ERC-6909 claim if a delivery fails. The force-exit is intended to succeed regardless of any action taken by the liquidity provider.

A position may have a subscriber, a contract chosen by the position owner that is notified when the position is unsubscribed or burned. The reported issue was that `unwindPosition` performed the unsubscribe inside a `PoolManager` unlock. The subscriber's `notifyUnsubscribe` callback could therefore interact with the unlocked `PoolManager` or restore its own approval, causing the burn that followed to revert, and could repeat this on every attempt.

The reviewed commit calls `_unsubscribe` directly from `unwindPosition` while the `PoolManager` is still locked, before any unlock is opened. The callback remains gas-capped and wrapped in `try/catch`, and any attempt to open an unlock or modify deltas from within it now reverts and is absorbed, and the approval required by the burn is only granted afterwards. Since `subscribe` is callable while the `PoolManager` is locked, the callback may still reattach a subscriber. In that case, `unwindPosition` force-clears the subscriber without notifying it and emits `Unsubscription`, so that `notifyBurn` is never called on an attacker-chosen contract during the burn. The now-redundant `UNSUBSCRIBE` action handler was removed.

While reviewing this change, a separate path was identified through which the liquidity provider can still block the force-exit without any subscriber, during the delivery of the unwound currencies.

Security Model and Trust Assumptions

- Adapter owners are trusted to use `unwindPosition` only for legitimate compliance purposes. The function burns the position without the consent of its owner.
- The force-exit is intended to be unblockable. Liquidity providers, their subscribers, and any currency paired with the permissioned token by the liquidity provider are treated as untrusted.
- The underlying permissioned token is trusted to enforce its own holder allowlist on transfer, which determines whether the liquidity provider or the adapter owner can receive the unwound permissioned currency.
- `unsubscribeGasLimit` is assumed to be set to a value that is sufficient for benign subscribers, while bounding the gas that a malicious subscriber can consume.

Privileged Roles

- **Adapter owner.** Set at creation by `PermissionsAdapterFactory` and transferable through `Ownable2Step`. Controls the adapter's allowlists and pause, and is the only account that may call `unwindPosition` for positions in pools containing its adapter. It is also the fallback recipient of a permissioned currency that cannot be delivered to the liquidity provider. Fully trusted.

Medium Severity

M-01 Recipients Can Block Force-Exit By Leaving an Unsettled Delta During Delivery

The `unwindPosition` function now unsubscribes with the `PoolManager` locked, so a subscriber callback can no longer leave a delta unsettled or revert the exit. However, the exit can still be blocked without any subscriber. Within the burn unlock, the `_unwindWithFallback` function delivers each currency by calling `poolManager.take` with the liquidity provider as the recipient. When the delivered currency hands control to the recipient, the liquidity provider can call `poolManager.mint` from within that callback, for example `mint(self, 0, 1)`. Since the `PoolManager` is still unlocked, the call succeeds and leaves a non-zero delta. This applies to native ETH, which is sent to the recipient's `receive` function through a low-level call, and to an ERC-20 token chosen by the liquidity provider whose `transfer` function reenters.

The `try/catch` around `take` does not prevent this, because `take` itself succeeds. The revert only occurs when the unlock closes and the `PoolManager` rejects the unsettled delta with `CurrencyNotSettled`, which is outside the `try` block. Since `hasSubscriber` returns `false`, the subscriber mitigation is skipped entirely. The attack can be repeated on every attempt and requires no capital or privileges, so the compliance force-exit can be blocked indefinitely. It is available to an EOA liquidity provider that pairs the permissioned token with its own malicious ERC-20 token, since `_mint` does not restrict the paired currency. It is also available to a smart-account liquidity provider that adds the delta-touching logic after minting, for example through an upgrade or EIP-7702 delegation, which cannot be detected by due diligence at mint time. Taking the currency to the position manager first and delivering it afterwards is not sufficient, since `take` still calls the token's `transfer` function inside the unlock.

Consider settling both currencies as ERC-6909 claims to the position manager within the burn unlock, since minting a claim executes no token code, and delivering each claim afterwards in its own isolated unlock. A recipient that reverts or leaves a delta unsettled would then only revert its own delivery, and the claim could be handed to the liquidity provider or, for a permissioned currency, to the admin through `poolManager.transfer`, which also executes no token code.

Update: *Resolved in pull request #602. Within the burn unlock, both currencies are now settled as ERC-6909 claims to the position manager. Each currency is then delivered in its own unlock with a gas cap set by the new* `deliveryGasLimit` *immutable. If a delivery fails, the claim is handed over instead.*

Notes & Additional Information

N-01 Inconsistent Order of Operations When Clearing a Subscriber

When a subscriber is reattached during the unsubscribe callback, the `unwindPosition` function force-clears it by first deleting the entry from the `subscriber` mapping and then calling `_setUnsubscribed`. However, the `_unsubscribe` function in `Notifier` performs the same two operations in the opposite order, first calling `_setUnsubscribed` and then deleting the mapping entry. The final state is identical in both cases, but the differing order makes the two code paths harder to compare.

Consider calling `_setUnsubscribed` before deleting the `subscriber` mapping entry in `unwindPosition`, so that it matches the order used in `Notifier._unsubscribe`.

Update: *Acknowledged, not resolved.*

Client Reported

CR-01 Liquidity Can Be Added Against an Unverified or Undeployed Permissions Adapter

The `_mint` function of `PermissionedPositionManager` requires at least one currency of the pool to be a verified permissions adapter. It then applies the hook allowlist and the owner's `LIQUIDITY_ALLOWED` permission through `_checkAllowedHook` and `_checkRecipientAllowed`. Both helpers skip any currency that is not a verified adapter, so the other currency of the pool is treated as an ordinary token. This also covers a permissions

adapter that has been created by `PermissionsAdapterFactory` but not yet verified, and an address with no code, such as the predictable address of an adapter the factory has not yet deployed. The same helpers are used by `_increase` and `_increaseFromDeltas`.

As a result, an account that is allowed to provide liquidity for a verified adapter A can open a position in an A/B pool, where B is either an unverified adapter or an undeployed adapter address. The position can be one-sided in A, so B never has to be paid. Once B is deployed and verified, swaps move the position into B, and its holder can withdraw B through the unchecked decrease and burn paths. At no point is the holder checked against B's `LIQUIDITY_ALLOWED` permission or B's hook allowlist.

Consider rejecting liquidity additions where a currency is a factory-created adapter that has not been verified, or a non-native currency address that has no code.

Update: *Resolved in pull request #603. The per-currency checks are now combined in `_validateLiquidityCurrency`, which `_mint`, `_increase` and `_increaseFromDeltas` all call. It reverts with `NoVerifiedAdapter` for an unverified adapter and with `NonContractCurrency` for a non-native currency address that has no code. A verified adapter must allow the pool's hook, and the position owner must hold its `LIQUIDITY_ALLOWED` permission.*

Conclusion

The reviewed changes remediate a previously reported issue in which a position's subscriber could block the compliance force-exit performed by `unwindPosition`. The fix unsubscribes while the `PoolManager` is locked, so the gas-capped `notifyUnsubscribe` callback can no longer interact with an open unlock, and force-clears any subscriber that the callback reattaches, so that `notifyBurn` is never called on an attacker-chosen contract during the burn.

The remediation closes the reported subscriber path. However, while reviewing it, one medium severity issue was identified through which the liquidity provider can still block the force-exit without any subscriber. The unwound currencies are delivered with `take` inside the burn unlock, so a recipient that gains control during the transfer, through native ETH or a paired ERC-20 token of its choosing, can leave an unsettled delta that reverts the entire unlock. No high or critical severity issues were identified.

Both issues share a root cause worth carrying forward. The force-exit is intended to succeed regardless of the liquidity provider's actions, which means that no code controlled by the liquidity provider can safely run while an unlock that must succeed is open. Subscriber callbacks, native ETH receipts, and the `transfer` functions of paired tokens all hand control to that party, and the `try/catch` blocks around them do not protect against a delta left unsettled, since that failure only surfaces when the unlock closes. Keeping the burn itself free of external calls into untrusted code, and isolating each delivery so that a failing recipient can only affect its own transfer, would make that invariant easier to maintain as the contract evolves.

The Uniswap team was very responsive throughout the review, providing prompt clarifications and delivering a fix for the reported issue within the engagement. We would like to thank them for their collaboration.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.